

CSA0904- Java Programming

CO2-AT1

Q1: Student Registration System

Suppose that a university system stores student data, where each student must be unique and can enroll in multiple courses.

- (a) Enumerate three Collection interfaces suitable for this system.
- (b) Design a Collection structure (diagram/representation) to store students and their enrolled courses.
- (c) Write a Java program using HashSet and HashMap with Generics to:
 - Add students
 - Map students to courses
 - Display data using Iterator

Code:

```
import java.util.*;
```

```
class Student {

    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public String toString() {
        return id + " " + name;
    }

    public static void main(String[] args) {

        HashSet<Student> students = new HashSet<>();

        Student s1 = new Student(101, "Alice");
        Student s2 = new Student(102, "Bob");

        students.add(s1);
        students.add(s2);
```

```

HashMap<Student, ArrayList<String>> map = new HashMap<>();

map.put(s1, new ArrayList<>(Arrays.asList("Java", "DBMS")));
map.put(s2, new ArrayList<>(Arrays.asList("Python", "CN")));

Iterator<Map.Entry<Student, ArrayList<String>>> it =
    map.entrySet().iterator();

while (it.hasNext()) {
    Map.Entry<Student, ArrayList<String>> e = it.next();
    System.out.println(e.getKey());
    System.out.println(e.getValue());
}
}
}

```

The screenshot shows the SIMATS Online Programming Lab interface. The header includes the SIMATS Engineering logo, the lab name, and the user's name (MATTA SWARNA AMBICA) with a unique ID. The main content area displays a question titled "Q1: Student Registration System" worth 10 marks. The question asks for three Collection interfaces, a Collection structure design, and a Java program using HashSet and HashMap with Generics. The "Your Code" section shows a Java program that defines a Student class and a main method that uses a HashSet to store students. The "Input" section shows "stdin input" and the "Output" section shows the expected output: "101 swarna [Java, DBMS]" and "102 ambica [Python, CN]".

Q2: E-Commerce Cart System

Suppose that an online shopping system maintains a cart where products are added, removed, and displayed in order.

- List three Collection classes applicable for cart management.
- Represent how cart items are stored using a List-based structure.
- Write a Java program using ArrayList and Iterator to:
 - Add items
 - Remove an item
 - Display all items

Code :

```
import java.util.ArrayList;
import java.util.Iterator;

public class ECommerceCart {
    public static void main(String[] args) {

        ArrayList<String> cart = new ArrayList<>();

        // Add items
        cart.add("Mouse");
        cart.add("Keyboard");
        cart.add("Monitor");
        System.out.println("Items added to cart successfully!");

        // Display items before removal
        System.out.println("\nCart items before removal:");
        Iterator<String> itr = cart.iterator();
        while (itr.hasNext()) {
            System.out.println("- " + itr.next());
        }

        // Remove an item using iterator
        itr = cart.iterator();
        while (itr.hasNext()) {
            String item = itr.next();
            if (item.equals("Mouse")) {
                itr.remove();
                System.out.println("\n'Mouse' removed from cart.");
            }
        }

        // Display items after removal
        System.out.println("\nCart items after removal:");
        Iterator<String> itr2 = cart.iterator();
        while (itr2.hasNext()) {
            System.out.println("- " + itr2.next());
        }
    }
}
```

SIMATS Online Lab
 SIMATS Online Programming Lab
 JAVA PROGRAMMING
 MATTA SWARNA AMBICA
 192472390KA

Q2: E-Commerce Cart System 10 marks

Suppose that an online shopping system maintains a cart where products are added, removed, and displayed in order.

(a) List three Collection classes applicable for cart management.
 (b) Represent how cart items are stored using a List-based structure.
 (c) Write a Java program using ArrayList and Iterator to:

- Add items
- Remove an item
- Display all items

Your Code

```

1 import java.util.ArrayList;
2 import java.util.Iterator;
3 public class ECommerceCart {
4     public static void main(String[] args) {
5         ArrayList<String> cart = new ArrayList<>();
6         cart.add("Mouse");
7         cart.add("Keyboard");
8         cart.add("Monitor");
9         System.out.println("Items added to cart successfully!");
10        System.out.println("\nCart items before removal:");
11        Iterator<String> itr = cart.iterator();
12        while (itr.hasNext()) {
13            String item = itr.next();
14            System.out.println("- " + item);
15        }
    }
}
  
```

Input
 stdin input

Output
 Items added to cart successfully!
 Cart items before removal:
 - Mouse
 - Keyboard
 - Monitor

Q1 Student Registration System 10 marks
 Q3 Library Management System 10 marks
 Q4 Employee Data Processing 10 marks
 Q5 Online Voting System 10 marks
 5/5 answered

Q3: Library Management System

Suppose that a library stores books with unique ISBN numbers and needs fast retrieval.

- (a) Name three Map implementations in Java.
- (b) Design a schema-like representation using Map for storing ISBN and book details.
- (c) Write a Java program using HashMap to:
- Insert book details
 - Retrieve a book
 - Display all books

Code:

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
public class LibraryManagementSystem {
```

```
    static class Book {
```

```
        String title;
```

```
        String author;
```

```
        String publisher;
```

```
        Book(String title, String author, String publisher) {
```

```
            this.title = title;
```

```
            this.author = author;
```

```
            this.publisher = publisher;
```

```

    }

    @Override
    public String toString() {
        return "Title: " + title +
            ", Author: " + author +
            ", Publisher: " + publisher;
    }
}

public static void main(String[] args) {

    HashMap<String, Book> library = new HashMap<>();

    // Insert book details
    library.put("978-0134685991", new Book("Effective Java", "Joshua Bloch", "Pearson"));
    library.put("978-0596009205", new Book("Head First Java", "Kathy Sierra", "O'Reilly"));
    library.put("978-0132350884", new Book("Clean Code", "Robert C. Martin", "Pearson"));

    // Retrieve a book
    String isbnToSearch = "978-0134685991";
    if (library.containsKey(isbnToSearch)) {
        System.out.println("Book found for ISBN " + isbnToSearch + ":");
        System.out.println(library.get(isbnToSearch));
    } else {
        System.out.println("No book found for ISBN " + isbnToSearch);
    }

    // Display all books
    System.out.println("\n--- Display All Books ---");
    for (Map.Entry<String, Book> entry : library.entrySet()) {
        System.out.println("ISBN: " + entry.getKey());
        System.out.println(entry.getValue());
        System.out.println();
    }
}
}

```

The screenshot shows the SIMATS Online Programming Lab interface. The top navigation bar includes the SIMATS logo, the lab name, and the user's name 'MATTA SWARNA AMBICA'. The main content area is titled 'Library Management System' and contains a question about a library system. The question asks for three Map implementations in Java, a schema-like representation using Map, and a Java program using HashMap to insert, retrieve, and display book details. Below the question is a 'Your Code' section with a Java code editor. The code defines a Book class with attributes title, author, and publisher, and a LibraryManagementSystem class with methods to insert, retrieve, and display book details. The output section shows the result of a search for ISBN 978-0134685991, displaying the book title 'Effective Java' by Joshua Bloch, published by Pearson.

Q4: Employee Data Processing (10 Marks)

Suppose that a company processes employee records and filters employees based on salary.

- List three advantages of Generics in Collections.
- Represent how employee data is stored using List with Generics.
- Write a Java program using Iterator to:
 - Store employee objects
 - Display employees with salary > 50,000

Code:

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
```

```
public class Employee {
    public static void main(String[] args) {
        List<EmployeeDetails> employees = new ArrayList<>();
        employees.add(new EmployeeDetails(101, "Swarna", 45000));
        employees.add(new EmployeeDetails(102, "Ambica", 60000));
        employees.add(new EmployeeDetails(103, "Cha", 75000));
        employees.add(new EmployeeDetails(104, "Dav", 40000));
```

```
        System.out.println("Employees with salary > 50,000:");
```

```
        Iterator<EmployeeDetails> iterator = employees.iterator();
```

```
while (iterator.hasNext()) {  
    EmployeeDetails emp = iterator.next();  
    if (emp.getSalary() > 50000) {  
        System.out.println(emp);  
    }  
}  
}
```

```
class EmployeeDetails {  
    private int empId;  
    private String name;  
    private int salary;  
  
    public EmployeeDetails(int empId, String name, int salary) {  
        this.empId = empId;  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public int getSalary() {  
        return salary;  
    }  
  
    @Override  
    public String toString() {  
        return "Employee ID: " + empId + ", Name: " + name + ", Salary: " + salary;  
    }  
}
```

Q5: Online Voting System (10 Marks)

Suppose that an online voting system ensures one vote per user and counts votes efficiently.

- Identify three suitable Collection types for this system.
- Design a structure using Set and Map for voters and vote counting.
- Write a Java program using HashSet and HashMap to:
 - Record votes
 - Prevent duplicate voting
 - Display results

Code:

```
import java.util.*;
```

```
public class OnlineVotingSystem {
    public static void main(String[] args) {
```

```
        // Set to prevent duplicate voting
        Set<String> votedUsers = new HashSet<>();
```

```
        // Map to store candidate -> vote count
        Map<String, Integer> voteCounts = new HashMap<>();
```

```
        // Initialize candidates
        voteCounts.put("Alice", 0);
        voteCounts.put("Bob", 0);
```



```

voteCounts.put("Charlie", 0);

// Cast votes (sample)
castVote("Voter101", "Alice", votedUsers, voteCounts);
castVote("Voter102", "Bob", votedUsers, voteCounts);
castVote("Voter101", "Charlie", votedUsers, voteCounts); // duplicate voter
castVote("Voter103", "Alice", votedUsers, voteCounts);
castVote("Voter104", "Charlie", votedUsers, voteCounts);
castVote("Voter105", "Bob", votedUsers, voteCounts);

// Display results
System.out.println("--- Voting Results ---");
for (Map.Entry<String, Integer> entry : voteCounts.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue() + " votes");
}
}

public static void castVote(String voterId, String candidate,
                           Set<String> votedUsers,
                           Map<String, Integer> voteCounts) {

    // Prevent duplicate voting
    if (votedUsers.contains(voterId)) {
        System.out.println("Error: Voter " + voterId + " has already cast a vote!");
        return;
    }

    // Validate candidate and count vote
    if (voteCounts.containsKey(candidate)) {
        votedUsers.add(voterId);
        voteCounts.put(candidate, voteCounts.get(candidate) + 1);
        System.out.println("Vote successfully cast by " + voterId + " for " + candidate);
    } else {
        System.out.println("Error: Candidate " + candidate + " not found.");
    }
}
}

```

